ClaimSense AI — Checkpoint 2 Final Submission

FDE Spine Capstone · Final Milestone Handover & Production Exit Pack

Participant Name:	**Amirdha Gopal Rajendran**	**Cohort Name:**	Forward Deployed Engineering LevelUp Program - Mid-August 2026
Tech Stack:	AWS · OpenAI Agents SDK / Ollama · FastAPI · Docker · Terraform	**Submission Stage:**	CHECKPOINT - 2 (FINAL)
Final Release Tag:	`v1.0.0` (Git Commit: `e21a5d6`)	**GitHub Repository:**	github.com/amirdhagopal/claimsense

1. Three Required Submission Assets (Summary)

All three capstone assets mandated for Checkpoint 2 are completed, verified, and linked below:

Required Asset	Location & Links	Status & Verification
Asset 1: Final Codebase	github.com/amirdhagopal/claimsense Release Tag: `v1.0.0`	39/39 TESTS PASSING Deterministic Spine, 4 Agents, BM25 Index, IaC
Asset 2: Observation Document	Checked in: `docs/observation-document.md` View on GitHub	ALL 6 SECTIONS COMPLETE Evals, Telemetry, Reliability Drill, Handover
Asset 3: Deployed Frontend & Demo Recording	**Frontend URL:** `http://localhost:8000/dashboard` **Video File:** `docs/ClaimSense_AI_Demo.mp4` Stream Video on GitHub	WEB APP & VIDEO VERIFIED Interactive UI, Real-time telemetry, 28s HD Video

2. Asset 1: Final Codebase Verification & Integrity

The ClaimSense AI codebase is fully functional, strictly typed, and completely validated across 5 development milestones:

- **Deterministic Spine Guarantee:** Identity, route-level authorization, policy effective date math, and deductible/total-loss arithmetic execute in pure compiled Python before any agent is invoked.
- **4-Agent Pipeline:** Implemented with typed Pydantic contracts (FileReaderAgent, CoverageAgent with BM25 citation retrieval, RiskAssessmentAgent, and DecisionDraftAgent).
- **Full Unit & Integration Suite:** `39 passed in 18.31s` covering API routes, token lifecycles, and two-user access control.
- **150-Case Evaluation Suite:** 150/150 cases validated with 100% agreement on standard claims and 0% adversarial leakage.

3. Asset 3: Deployed Frontend & Demo Recording Details

🎥 Demo Recording Video Verification

A full video demonstration of the working application has been rendered and committed to the repository:

Direct Video File:	`docs/ClaimSense_AI_Demo.mp4` (497 KB, 1440x900 HD, 24fps H.264 video)
GitHub Stream Link:	https://github.com/amirdhagopal/claimsense/blob/main/docs/ClaimSense_AI_Demo.mp4
Animated Preview:	`docs/assets/ClaimSense_AI_Demo.gif` (embedded in README and documentation)
Live Web Co-Pilot URL:	`http://localhost:8000/dashboard` (served via FastAPI & static dark-mode UI)

Video Walkthrough Timestamp Index:

Timestamp	Workflow Step	What Is Demonstrated
0:00 – 0:04	Workbench Overview	Claims Adjuster Workbench, claim `CLM-2026-0101` metadata, live CloudWatch SLA cards.
0:04 – 0:08	AI Adjudication	1ms deterministic spine check, FileReader facts extraction, BM25 clause retrieval (`CLS-COV-001`).
0:08 – 0:12	Observability Dashboard	Live telemetry metrics: call volume (120 calls), error rate (5.83%), p95 latency (1570 ms), SLA status.
0:12 – 0:16	Trust Boundary	Two-user access control: User A (Jane, assigned) allowed; User B (Bob, unassigned) refused with 0% leakage.
0:16 – 0:20	Audit Trail	Immutable cryptographic audit log with user ID, action, resource, SHA-256 hash, and millisecond timestamps.
0:20 – 0:28	150-Case Evals	150 benchmark cases: 49 Approve, 40 Deny, 32 Flag, 29 Route; 33 adversarial attacks successfully blocked.

4. Google Form Field Values (Quick Reference)

To submit Checkpoint 2 on the Google Form, copy and paste the following verified inputs:

Name *	**Amirdha Gopal Rajendran**
Cohort Name *	**Forward Deployed Engineering LevelUp Program - Mid-August 2026**
Tech Stack Used *	AWS + OpenAI Agents SDK / Ollama + Python 3.12 (FastAPI) + Docker + Terraform + Vanilla Web UI
Checkpoint Selection *	**Checkpoint - 2**
Submission Link (File) *	Upload this PDF: `docs/ClaimSense_AI_Checkpoint_2_Final_Submission.pdf`

5. Asset 2: Completed Observation Document (Executive Summary)

The observation document (`docs/observation-document.md`) provides a rigorous technical retrospective across all 6 required areas:

1. Production Deployment & Readiness Assessment

Containerized using Docker and orchestrated for AWS ECS Fargate with KMS encryption and ALB. The architecture enforces an inviolable separation: all access control, date math, and money calculations execute deterministically before the LLM pipeline is invoked.

2. Evaluation Suite Results (150 Cases)

Evaluated on 150 diverse synthetic claims: 49 Approvals, 40 Denials, 32 Referrals (Fraud), and 29 Routing cases. Crucially, the suite includes **33 hostile adversarial attacks** across 4 distinct threat families: prompt injection (8), cross-record leakage (9), forced auto-decision (8), and narrative contradictions (8). The pipeline achieved **100% defense** with zero unauthorized data exposure.

3. Observability, Telemetry & SLA Compliance

Instrumented with an in-memory & CloudWatch metrics pipeline tracking invocation volume, error rate, and p50/p95/p99 latencies. Configured with automated CloudWatch alarms (`HighLatencyP95Alarm` > 60s, `HighErrorRateAlarm` > 5%). Average local p95 latency is 1,570 ms—substantially outperforming the 60,000 ms SLA.

4. Platform Reliability & Failure Modes (SEV-2 Incident Drill)

Conducted a live regression drill simulating an upstream model timeout breach. The CloudWatch alarm triggered within 30 seconds, initiating an automated ECS task rollback. Full service recovery was verified with a **Mean Time to Resolution (MTTR) of 2 minutes 45 seconds** (detailed in `docs/incident-report.md`).

5. Operational Handover & Production Runbook

Authored 4 complete Standard Operating Procedures (SOPs) in `docs/runbook.md` enabling independent operators to run the system: SOP-01 (Provisioning Users), SOP-02 (Rolling Back Deployments), SOP-03 (Escalating to SIU), and SOP-04 (Updating Policy Corpus & Re-indexing BM25).

6. Engineering Retrospective & Lessons Learned

Key takeaways include: (a) deterministic spines eliminate financial hallucinations; (b) smaller foundation models require strict prompt sanitization and JSON fence stripping; and (c) human-in-the-loop write-back is the most effective safeguard for high-stakes insurance adjudication.